

Лабораторная работа 2

Тема: Репозиторий в Spring Data JPA

Цель: изучить способ использования репозитория в Spring Data JPA

Теоретическая часть

В данной работе рассмотрим использование СУБД H2 с Spring Boot. Как и в случае с другими базами данных, в экосистеме Spring Boot имеется полная встроенная поддержка.

Начнем с зависимостей *h2* и *spring-boot-starter-data-jpa*:

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Конфигурация базы данных

По умолчанию Spring Boot настраивает приложение для подключения к хранилищу в памяти с именем пользователя *sa* и пустым паролем.

Однако мы можем изменить эти параметры, добавив следующие свойства в файл *application.properties* :

```
spring.datasource.url=jdbc:h2:mem:institut
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
```

База данных в процессе работы располагается только в памяти, что приводит к потере данных после перезапуска приложения.

Если необходимо иметь сохраненную на диске БД, то необходимо изменить это поведение, используя файловое хранилище. Для этого нужно обновить свойство *spring.datasource.url*:

```
spring.datasource.url=jdbc:h2:file:/data/institut
```

Выполнение операций CRUD с H2 в Spring Boot такое же, как и с другими базами данных SQL, поэтому дальнейший материал может быть применен к любой другой реляционной СУБД.

Инициализация источника данных

Spring Boot позволяет управлять изменениями в базе данных. Если оставить конфигурацию по умолчанию, она будет искать сущности в пакетах и автоматически создавать соответствующие таблицы.

Но иногда требуется более тонкий контроль над изменениями в базе данных. Если имя таблицы отличается от имен класса, то можно указать это с помощью аннотации:

```
@Table(name = "students")
```

Рассмотрим на примере сущности студент.

```
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;
    @Column(name = "name", nullable = false)
    private String name;
    private int age;
    //...getters and setters
}
```

Если мы запустим приложение, Spring Boot создаст пустую таблицу, но ничем ее не заполнит.

Если необходимо реализовать базовые сценарии SQL для инициализации базы данных, то для этого можем использовать файлы *data.sql* и *schema.sql* в Spring.

Чтобы продемонстрировать это, добавим файл *data.sql* в каталог *src/main/resources*

```
INSERT INTO students (name, age) VALUES ('Михаил', 20);
INSERT INTO students (name, age) VALUES ('Мария', 19);
INSERT INTO students (name, age) VALUES ('Магомед', 21);
```

Здесь скрипт заполняет таблицу студентов в нашей схеме некоторыми примерами данных.

Spring Boot автоматически найдет этот файл и запустит его. Это хороший способ заполнить базу данных для целей *тестирования* или *инициализации*.

Мы можем отключить это поведение по умолчанию, установив для свойства `spring.sql.init.mode` значение `never`.

По умолчанию скрипт *data.sql* выполняется перед инициализацией Hibernate. Это согласовывает инициализацию на основе сценариев с другими инструментами миграции баз данных, то есть если БД уже имеется. Поскольку в данной работе мы каждый раз воссоздаем схему, сгенерированную Hibernate, нужно установить дополнительное свойство:

```
spring.jpa.defer-datasource-initialization=true
```

Это изменяет поведение Spring Boot по умолчанию и заполняет данные после того, как схема сгенерирована Hibernate. Кроме того, можно использовать сценарий *schema.sql* для создания схемы, сгенерированной Hibernate, до заполнения с помощью *data.sql*. Однако такое смешение различных механизмов генерации схем не рекомендуется.

Например, в файл *schema.sql* может быть прописан такой скрипт:

```
CREATE TABLE students (
    id INTEGER NOT NULL AUTO_INCREMENT,
    name VARCHAR(128) NOT NULL,
```

```
PRIMARY KEY (id)
);
```

Spring возьмет этот файл и будет использовать его для создания схемы.

Обратите внимание, что инициализация на основе сценариев, т. е. через *schema.sql* и *data.sql*, и инициализация Hibernate при работе совместно могут вызывать конфликты.

Для решения таких конфликтов модно отключить автоматическое создание схемы Hibernate:

```
spring.jpa.hibernate.ddl-auto=none
```

Это гарантирует, что инициализация на основе сценария выполняется напрямую с использованием *schema.sql* и *data.sql*.

Если мы по-прежнему хотим иметь автоматическую генерацию схемы Hibernate в сочетании с созданием схемы на основе сценариев и заполнением данных, необходимо использовать:

```
spring.jpa.defer-datasource-initialization=true
```

Это гарантирует, что после создания схемы Hibernate дополнительно считывается *schema.sql* для любых дополнительных изменений схемы, а *data.sql* выполняется для заполнения базы данных.

Кроме того, инициализация на основе сценариев выполняется по умолчанию только для встроенных баз данных, чтобы всегда инициализировать базу данных с помощью сценариев, необходимо использовать:

```
spring.sql.init.mode=always
```

Доступ к консоли H2

База данных H2 имеет встроенную консоль с графическим интерфейсом для просмотра содержимого базы данных и выполнения запросов SQL. По умолчанию консоль H2 не включена в Spring.

Чтобы включить его, нужно добавить следующее свойство в *application.properties* :

```
spring.h2.console.enabled=true
```

Затем, после запуска приложения, можно перейти к <http://localhost:8080/h2-console>, что предоставит нам страницу входа.

На странице входа используются те же учетные данные, которые записаны в *application.properties*:

Login

Saved Settings: Generic H2 (Embedded) [dropdown]

Setting Name: Generic H2 (Embedded) [Save] [Remove]

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:institut

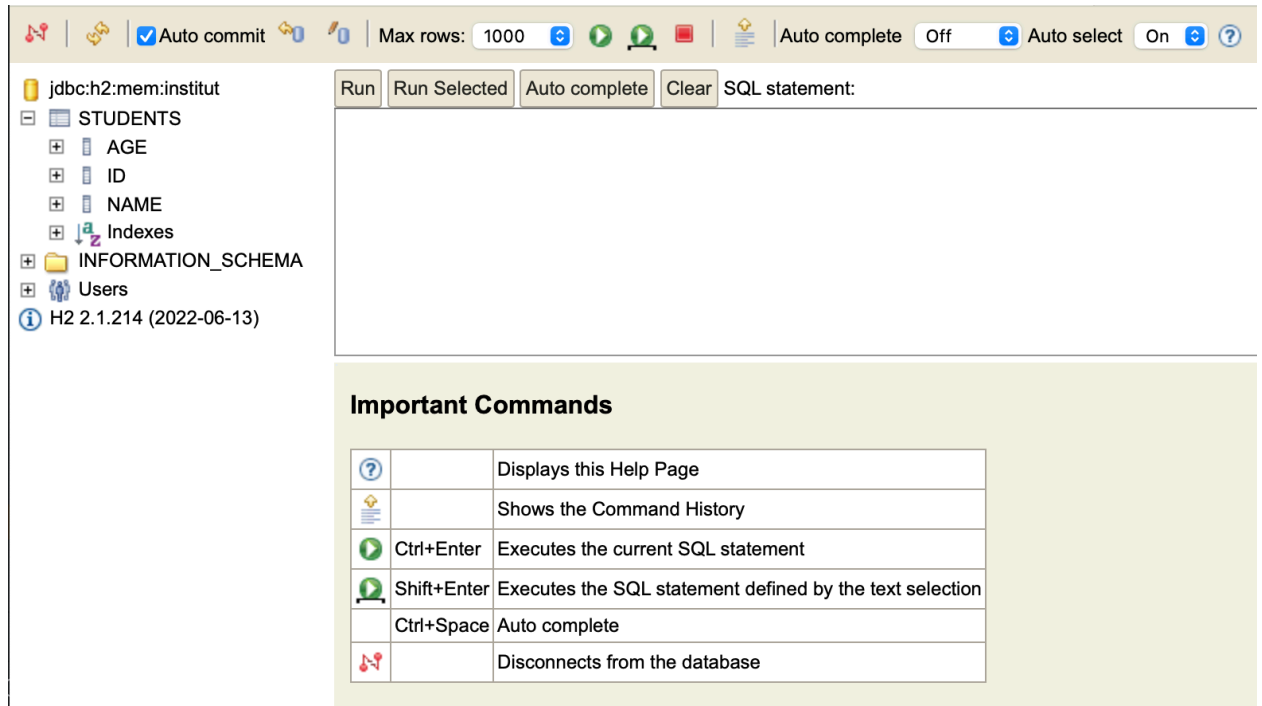
User Name: sa

Password: [input field]

[Connect] [Test Connection]

Как только мы подключимся, увидим веб-страницу со списком всех таблиц в левой части страницы и текстовое поле для выполнения SQL-запросов.

Веб-консоль имеет функцию автозаполнения, которая предлагает ключевые слова SQL. Тот факт, что консоль невелика, делает ее удобной для визуального осмотра базы данных или непосредственного выполнения необработанного SQL.



Более того, мы можем дополнительно настроить консоль, указав следующие свойства в *application.properties* проекта с нужными нам значениями:

```
spring.h2.console.path=/h2-console
spring.h2.console.settings.trace=false
spring.h2.console.settings.web-allow-others=false
```

В приведенных выше фрагментах мы установили путь к консоли как */h2-console*, что соответствует адресу и порту нашего работающего приложения. Поэтому, если наше приложение запущено по адресу `http://localhost:8080`, консоль будет доступна по адресу `http://localhost:8080/h2-console`.

Кроме того, можно установить `spring.h2.console.settings.trace` значение `false`, чтобы предотвратить вывод трассировки, и мы также можем отключить удаленный доступ, установив `spring.h2.console.settings.web-allow-others` в `false`.

Регистрация запросов JPA на стандартный вывод

Spring JDBC и JPA предоставляют абстракции над собственными API JDBC, позволяя разработчикам отказаться от собственных запросов SQL. Однако нам часто нужно видеть эти автоматически сгенерированные SQL-запросы и порядок их выполнения для целей отладки.

Самый простой способ вывести запросы на стандартный вывод — добавить в *application.properties* следующее:

```
spring.jpa.show-sql=true
```

В таком случае, для нашего примера в консоли получим скрипты создания таблиц:

```
Hibernate: drop table if exists students cascade
Hibernate: create table students (age integer not null, id
varchar(255) not null, name varchar(255), primary key (id))
```

Чтобы напечатать SQL в форматированном виде, мы можем добавить:

```
spring.jpa.properties.hibernate.format_sql=true
```

Хотя это чрезвычайно просто и удобно, это не рекомендуется, так как напрямую выгружает все в стандартный вывод без какой-либо оптимизации структуры ведения журнала. Более того, он не регистрирует параметры подготовленных операторов.

Используйте только на этапе разработки и отладки приложения.

Создание репозитория

Repository — описанный в Spring Data интерфейс, играющий роль удобной абстракции для различных баз данных. Это несколько интерфейсов которые используют JPA Entity для взаимодействия с ней.

Он принимает класс домена для управления, а также тип идентификатора класса домена в качестве аргументов типа. Этот интерфейс действует в основном как маркерный интерфейс для получения типов для работы и помогает вам находить интерфейсы, расширяющие его.

Базовый репозиторий, определенный в Spring JPA – это CrudRepository:

```
public interface
    CrudRepository<T, ID extends Serializable>
        extends Repository<T, ID>
```

Создадим в проекте отдельный пакет repository, а в нем интерфейс StudentRepository, который наследуется от CrudRepository. В качестве класса домена укажем наш класс Student, в качестве типа ключа – Long.

```
public interface StudentRepository
    extends CrudRepository<Student, Long> {
}
```

Теперь в мы можем создать экземпляр нашего репозитория и использовать определенные в нем методы.

Например, в классе контроллера:

```
private final StudentRepository studentRepository;
```

В конструкторе необходимо будет его инициализировать полученным из контекста приложения репозиторием, созданным автоматически:

```
public MainController(StudentRepository studentRepository) {
    this.studentRepository = studentRepository;
}
```

Если не нужна инициализация в конструкторе, то можно доверить это средствам самого Spring:

```
@Autowired
private StudentRepository studentRepository;
```

Если нужно получить экземпляр студента по известному *id*, то можно воспользоваться методом `findById(id)`. Метод вернет экземпляр `Optional`, в котором будет найденный экземпляр `Student`:

```
Optional<Student> optionalStudent =  
    studentRepository.findById(id);
```

Сам *id* будем получать из адресной строки. Укажем это `GetMapping`:

```
@GetMapping("/details/{id}")
```

Далее в списке параметров метода необходимо указать аннотацию `@PathVariable()`, которой указать имя параметра. Самих параметров в адресной строке может быть несколько, поэтому для каждого из них указывается отдельная переменная.

Пример метода контроллера, с получением параметра:

```
public String details(Model model, @PathVariable("id") String id)
```

Далее уже в теле метода путем вызова метода `findById()` получаем данные из БД.

Пример метода получения информации о студенте:

```
@GetMapping("/details/{id}")  
public String details(Model model, @PathVariable("id") Long id) {  
    Optional<Student> optionalStudent =  
        studentRepository.findById(id);  
    if (optionalStudent.isEmpty()) {  
        return "redirect:/students/main";  
    }  
    model.addAttribute("selectedStudent", optionalStudent.get());  
    return "details";  
}
```

Аналогично, можно работать с другими методами, требующих параметров.

Список остальных методов можно посмотреть у класса `CrudRepository`.

Если необходимо, чтобы все запросы начинались с одного и того же пути, то для контроллера можно указать аннотацию `@RequestMapping()`, у которой записать имя префикса:

Например, `@RequestMapping("/students")`

Редактирование информации

При создании нового студента, мы передавали на форму пустой экземпляр. Далее форма методом `post` возвращала заполненный экземпляр.

При редактировании подход аналогичный, только вместо пустого студента будем отдавать текущего выбранного.

Форма представления может использоваться та же самая, что и для создания нового, либо сделать другую. Разница будет в том, что при создании нового студента нам не нужно поле *id*. А при редактировании это поле обязательно должно быть в экземпляре.

Пример реализации методов редактирования информации:

```
@GetMapping("/update/{id}")  
public String editStudent(Model model,  
    @PathVariable("id") Long id) {  
    Optional<Student> optionalStudent =
```

```

        studentRepository.findById(id);
    if (optionalStudent.isEmpty()) {
        return "redirect:/students/main";
    }
    model.addAttribute("student", optionalStudent.get());
    return "edit_student";
}

```

Пример страницы редактирования информации с скрытым полем id (обратите внимание на скрытое поле):

```

<form action="#" th:action="@{/students/update}"
    th:object="${student}" method="post">
    <input type="hidden" th:field="*{id}"/>
    <p>Имя студента:
        <input type="text" placeholder="Введите имя"
            th:field="*{name}"/></p>
    <p>Возраст:
        <input type="text" placeholder="Введите возраст"
            th:field="*{age}"/></p>
    <p><input type="submit" value="Отправить"/>
        <input type="reset" value="Сброс"/></p>
</form>

```

Получение данных с формы для записи в базу:

```

@PostMapping("/update")
public String editStudent(@ModelAttribute Student student,
    Model model) {
    //TODO сделать проверку полученного экземпляра
    studentRepository.save(student);
    return "redirect:/students/main";
}

```

Удаление объекта из базы

Для реализации удаления экземпляра из БД в репозитории есть готовый метод `deleteById(T id)`. Перед его вызовом необходимо убедиться, что такая запись существует в БД:

```

@GetMapping("/delete/{id}")
public String delete(Model model, @PathVariable("id") Long id) {
    if (studentRepository.existsById(id)) {
        studentRepository.deleteById(id);
    }
    return "redirect:/students/main";
}

```

Отображение списков с использованием Thymeleaf

При обращении к точке (endpoint) */institut* будем возвращать представление «*institut.html*»:

```
return "institut";
```

в которое передадим не один объект, а сразу целый список:

```
model.addAttribute("students", allStudents);
```

Этот список предварительно нужно запросить у репозитория:

```
List<Student> allStudents = studentRepository.findAll();
```

Полный метод выглядит следующим образом:

```
@GetMapping("/institut")
public String mainPage(Model model) {
    List<Student> allStudents = studentRepository.findAll();
    model.addAttribute("students", allStudents);
    return "main";
}
```

Отображать список студентов будем в таблице. Размер таблицы заранее неизвестен, поэтому воспользуемся тегом `each`, который предоставляет шаблонизатор.

В циклах можно использовать java-объекты, реализующие интерфейсы `java.util.Iterable`, `java.util.Map`, а также массивы. Синтаксис `th:each` включает 2 переменные: переменная элемента (*item variable*) и переменная состояния (*state variable*).

Синтаксис:

```
<someHtmlTag th:each="item : ${items}">
    ....
</someHtmlTag>
```

В данном случае тег `th:each` продублирует содержимое тега `someHtmlTag` столько раз, сколько элементов в коллекции `items`. При этом, в каждой итерации будет доступен элемент `item`, представляющий текущий элемент коллекции.

Если необходимо повторить элементы, не являющиеся содержимым какого-либо тега, то можно использовать виртуальный тег.

Тег `<th:block>` – это виртуальный тег в Thymeleaf, он не соответствует никакому тегу HTML.

```
<th:block th:each="item : ${items}">
    ....
</th:block>
```

Переменная состояния (*State variable*) это объект, содержащий информацию текущего состояния цикла

Index – Текущий индекс повтора (*iteration*), начиная с 0.

Count – Количество обработанных элементов до настоящего момента.

size – Сумма элементов в списке.

even/odd – Проверяет текущий индекс (*index*) повтора (*iteration*) является четным или нечетными.

first – Проверяет является ли настоящий повтор первым или нет.

last – Проверяет является ли настоящий повтор последним или нет.

Пример создания таблицы на основе списка студентов:

```
<tr th:each="student : ${students}">
    <td th:text="${student.id}"/>
    <td th:text="${student.name}"/>
    <td th:text="${student.age}"/>
    <td><a class="btn btn-secondary" th:href="@{'/details/'
        + ${student.id}}">Details</a></td>
    <td><a class = "btn btn-danger"
        th:href = "@{'/students/delete/'
        + ${student.id}}">Delete</a></td>
</tr>
```

Заголовок файла стандартный для всех таких файлов. Также в начале сформируем заголовок таблицы.

Полный файл:

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <title>Application</title>
</head>

<body>
<div class="container">
    <h1>Институт</h1>
    <span th:if="${students.size() == 0}">Список студентов пустой</span>
    <table th:if="${students.size() > 0}" class="table table-hover">
        <thead class="thead-dark">
            <tr>
                <td>ID</td>
                <td>Имя</td>
                <td>Возраст</td>
                <td></td>
                <td></td>
            </tr>
        </thead>
        <tbody>
            <tr th:each="student : ${students}">
                <td th:text="${student.id}"/>
                <td th:text="${student.name}"/>
                <td th:text="${student.age}"/>
                <td><a class="btn btn-secondary" th:href="@{'/details/' +
${student.id}}">Details</a></td>
                <td><a class="btn btn-danger" th:href="@{'/students/delete/' +
${student.id}}">Delete</a></td>
```

```

        </tr>
    </tbody>
</table>
</div>
</body>
</html>

```

Для получения конкретного объекта по его *id*, сделаем соответствующий метод, передающий полученный идентификатор в сервис:

```

@GetMapping("/details/{id}")
public String detailsPage(Model model, @PathVariable("id") Long id) {
    Student selectedStudent = studentRepository.findById(id);
    model.addAttribute("selectedStudent", selectedStudent);
    return "details";
}

```

В параметрах метода использована дополнительная аннотация `@PathVariable`, указывающая на то, что данный параметр получается из адресной строки. Аннотация `@PathVariable` связывает часть URL с параметром метода контроллера.

Spring автоматически конвертирует значения в нужный тип (`int`, `Long`, `String` и т.д.). Если конвертация невозможна, будет ошибка *400 Bad Request*.

В данном примере, если пользователь перейдет по адресу `/details/5`, то переменная *id* получит значение 5.

За отображение детальной информации будет отвечать файл `«details.html»`. В этом файле с помощью тега `th:text` получим поля *id*, *name* и *age* из переданного объекта.

```

<body>
<div class="container">
    <h1>Details</h1>
    <span th:text="${selectedStudent.id + ' ' + selectedStudent.name +
    ' ' + selectedStudent.age}" />
</div>
</body>

```

Аналогично, в контроллере реализуем страницу удаления студента (через запрос GET, вместо DELETE). Только по итогу метода будем выполнять переадресацию на страницу *«institut»*

```

@GetMapping("/students/delete/{id}")
public String deleteStudentById(@PathVariable("id") Long id) {
    studentRepository.deleteById(id);
    return "redirect:/institut";
}

```

Задание на лабораторную работу

1. Рассмотреть пример создания БД с таблицей *Студент*.
2. Дополнить информацию о студенте полями: *фамилия*, *отчество*, *город*, *пол*, *возраст*, *группа*, *факультет*.

3. Реализовать приложение с функциями создания, редактирования, удаления и отображения в соответствии с вариантом.
4. Реализовать методы получения списка всех сущностей, одной сущности по id, удаление по id, добавление и изменение.
5. При поиске сущности, изменении или удалении проверять её существование в БД.
6. При отображении страниц использовать стили оформления.

Вариант	Задание
1 «Человек»	фамилия; имя; отчество; пол; национальность; рост; вес; дата рождения (год, месяц число); номер телефона.
2 «Школьник»	фамилия; имя; отчество; пол; национальность; рост; вес; дата рождения (год, месяц число); номер телефона; домашний.
3 «Студент»	фамилия; имя; отчество; пол; национальность; рост; вес; дата рождения (год, месяц число); номер телефона; ВУЗ; курс; группа; средний бал; специальность.
4 «Покупатель»	фамилия; имя; отчество; пол; национальность; рост; вес; дата рождения (год, месяц число); номер телефона; номер кредитной карточки; банковского счета.
5 «Пациент»	фамилия; имя; отчество; пол; национальность; рост; вес; дата рождения (год, месяц число); номер телефона; номер больницы; отделение; номер медицинской карты; диагноз; группа крови.
6 «Владелец автомобиля»	фамилия; имя; отчество; номер телефона; марка автомобиля; номер автомобиля; номер техпаспорта.
7 «Военнослужащий»	фамилия; имя; отчество; национальность; дата рождения (год, месяц число); должность; звание.
8 «Рабочий»	фамилия; имя; отчество; национальность; дата рождения (год, месяц число); No цеха; табельный номер; образование; год поступления на работу.
9 «Владелец телефона»	фамилия; имя; отчество; домашний адрес (почтовый индекс, страна, область, район, город, улица, дом, квартира); No телефона.
10 «Абитуриент»	фамилия; имя; отчество; пол; национальность; дата рождения (год, месяц число); оценки по экзаменам; проходной балл.